



String Hashing

- Priyansh Agarwal



Why String Hashing?

- Optimizing Brute Force solutions
- Comparing 2 strings ==
- Daddy of String Algos kmp, z

TLE TLE TLE TLE

TLE

$x \longrightarrow \text{hash}(x) \longrightarrow$ unique integer that
we can store



Requirements

- If $a == b$, $\text{Hash}(a) == \text{Hash}(b)$ trivial
- If $a \neq b$, $\text{Hash}(a) \neq \text{Hash}(b)$
 - If this is true with a very high probability we are good to go
- Calculating Hash function should be fast enough
- Hash of a string shouldn't change in the code
- Hash value should be itself $O(1)$

strings are hard to play around with

A
(n)

B
(n)

x $\longrightarrow$ hash(x) $\longrightarrow$ int
(string)

is it possible to convert every string in
the world to a unique integer

Yes

$$297 \Rightarrow 2 \cdot 10^2 + 9 \cdot 10^1 + 7 \cdot 10^0$$

unique representation of any integer

$$x_1 \cdot 10^0 + x_2 \cdot 10^1 + x_3 \cdot 10^2 + \dots + x_n \cdot 10^{n-1}$$

$$0 \leq x_i < 10$$

$$A \rightarrow \{x_1, x_2, \dots, x_n\}$$

$$B \rightarrow \{y_1, y_2, \dots, y_m\}$$

$$A \stackrel{!}{=} B \quad \text{if} \quad \underline{\underline{n \stackrel{!}{=} m}} \quad \text{or} \quad \text{any } x_i \stackrel{!}{=} y_i$$

for strings $\rightarrow A \neq B$ if

$\text{len}(A) \neq \text{len}(B)$ or $A[i] \neq B[i]$
for any
 i

integers

$$x_1, x_2, \dots, x_n \rightarrow \boxed{0 \leq x_i < 10}$$

$$x_1 \cdot 10^n + x_2 \cdot 10^{n-1} + \dots + x_n \cdot 10^1$$

strings (lower case)

$$x_1, x_2, \dots, x_n \rightarrow a \leq x_i \leq z$$

$a \rightarrow 0$, $b \rightarrow 1$, $c \rightarrow 2$ $z \rightarrow 25$

$A \rightarrow$ $\{x_1, x_2 \dots x_n\}$
(string)

$B \rightarrow$ $\{1+x_1 - 'a', \underline{1+x_2 - 'a'}, \dots, 1+x_n - 'a'\}$
(int arr)

$$1 \leq B[i] \leq 26$$

for integers

$$\{x_1, x_2, \dots, x_n\}$$



$$x_1 \cdot 10^{n-1} + x_2 \cdot 10^{n-2} + \dots + x_n \cdot 10^0$$

for strings
(lower case)

$$\{x_1, x_2, \dots, x_n\}$$

$$y_i = |x_i - 'a'|$$

$$\{y_1, y_2, \dots, y_n\}$$

$$1 \leq y_i \leq 26$$

$$\{y_1 \cdot 26^{n-1} + y_2 \cdot 26^{n-2} + \dots + y_n \cdot 26^0\}$$

$$a \rightarrow 0$$

$$aa \rightarrow 0.26^0$$

$$aaa \rightarrow 0.26^1 + 0.26^0$$

$$aaaa \rightarrow 0.26^2 + 0.26^1 + 0.26^0$$

$$1 \neq 11 \neq 111 \neq 1111$$

$$000 = 0 = 00 = 00000$$

$$001 = 00001$$

$$aa = 1 \cdot 26^1 + 1 \cdot 26^0 = 27$$

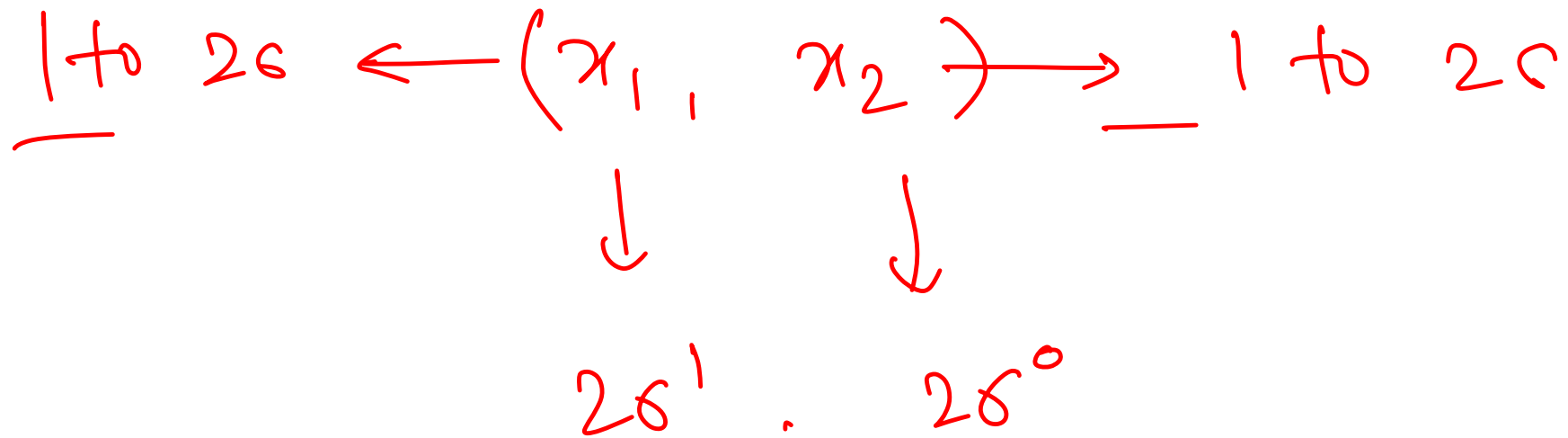
$$z = 26 \cdot 26^0 = 26$$

$$az = 1 \cdot 26^1 + 26 \cdot 26^0 = 52$$

$$as = 1 \cdot 26^1 + 2 \cdot 26^0 = 28$$

$$aaa = 1 \cdot 26^2 + 1 \cdot 26^1 + 1 \cdot 26^0$$

A $\rightarrow$ of length 2



$$\text{highest} = 26^2 + 26$$

$$\text{lowest} = 26 + 1$$

$$\text{range} = 27 \text{ to } \underline{\underline{26(27)}}$$

A of length 3

$$\begin{array}{ccc} (x_1 & x_2 & x_3) \\ \downarrow & \downarrow & \downarrow \\ 26^2 & 26^1 & 26^0 \end{array}$$

$$\text{highest} = 26 \cdot 26^2 + 26 \cdot 26^1 + 26 \cdot 1$$

$$\text{lowest} = 1 \cdot 26^2 + 1 \cdot 26 + 1 \cdot 26^0$$

$$\text{lowest} = \underline{\underline{26^2 + 26 + 1}} = \underline{\underline{26(27) + 1}}$$

$$\begin{aligned} \text{highest} &= 26^3 + 26^2 + 26 \\ &= 26(26^2 + 26 + 1) \end{aligned}$$

$$= 26(26(27) + 1)$$

for 2 length

26 + 1 to 26(27)

for 3 length

26(27) + 1 to 26(26(27) + 1)

$$26 = 26 \cdot 26^1 + 2$$

$$008 = 1 \cdot 26^2 + 1 \cdot 26^1 + 2 \cdot 26^0$$

$$A \rightarrow x_1 \cdot \underline{50}^{n-1} + x_2 \cdot 50^{n-2} + \dots + x_n \cdot 50^0$$



5120

2 len

3 len



+1

huge diff



String



Unique integer

Can you store every
integer

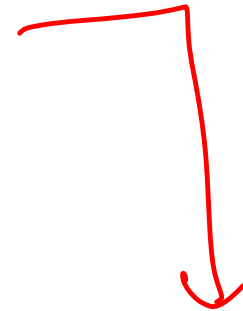
infinite

String



infinite

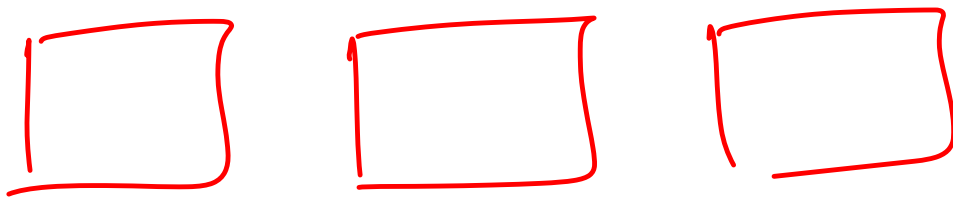
Unique integer



Something that
we can store

Cannot have
more than

$\leq 10^{18}$



3 bones

4 Sally

1 son must contain at least 2
Sally

x

integers

y

$\%$

$x \% 10$



0 to 9

$y \% 10$



0 to 9

$$P(X \% 10 = 4 \% 10)$$

$$= P(X \% 10 = 4 \% 10 = 0)$$

$$+ P(X \% 10 = 4 \% 10 = 1)$$

⋮
+

$$P(X \% 10 = 4 \% 10 = 9)$$

$$= \frac{1}{10} \cdot \frac{1}{10} + \frac{1}{10} \cdot \frac{1}{10} + \dots + \text{lots of times} = \frac{1}{10} \cdot \frac{1}{10} \cdot 10$$

$$= \frac{1}{10}$$

① X and y are random nos

$$P(X \% 10 = y \% 10) = 1/10$$

② X and y are random nos sat

we know $X = y$

$$P(X \% 10 = y \% 10) = 1$$

if X and Y are random hyper
numbers

$$P(\underline{X \% M} = \underline{Y \% M}) = \frac{1}{M}$$

string A, string B

$$A \neq B$$

hash, (x) = $\overset{\text{string}}{(x_0 - 'a' + 1)} \cdot 26^{n-1} +$

$$(x_1 - 'a' + 1) \cdot 26^{n-2} \dots (x_{n-1} - 'a' + 1) \cdot 26^0$$

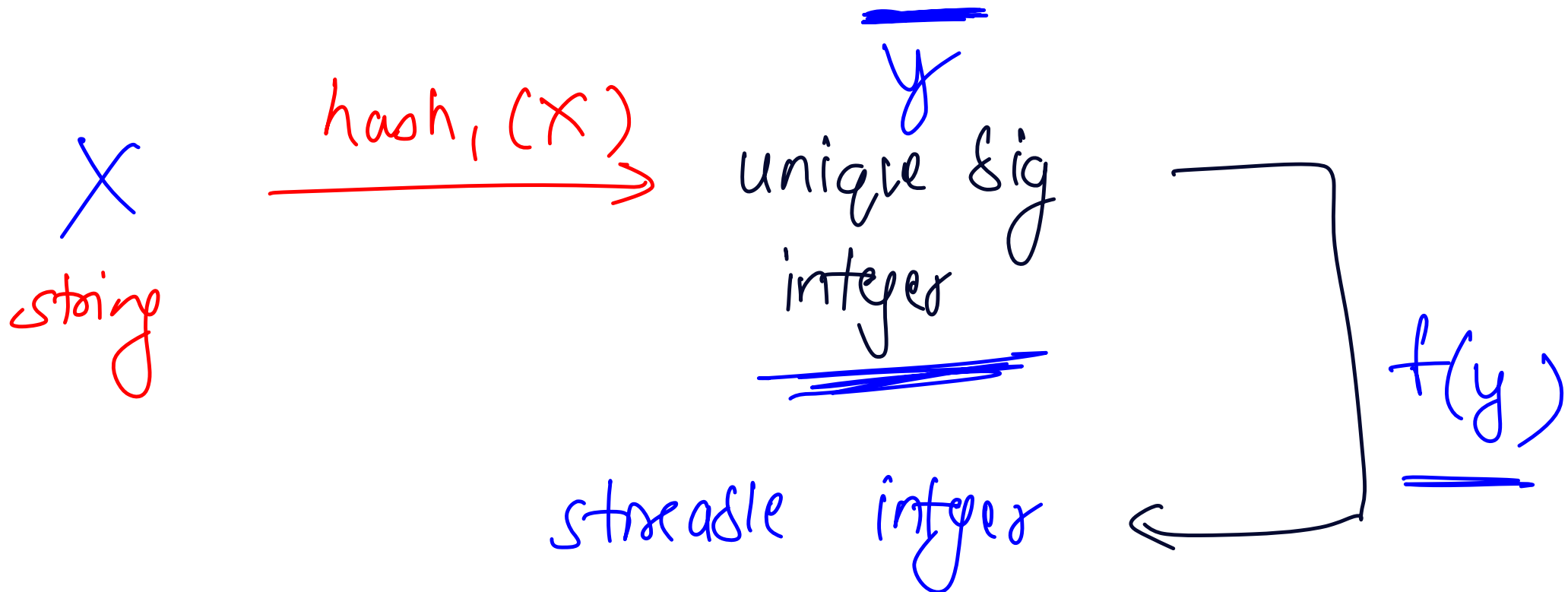
$$\underline{\underline{f(y)}} = \underline{\underline{y \% 10^{15}}}$$

$$0 \leq f(\text{hash}_i(x)) < 10^{15}$$

string A $\neq$ string B

$\text{hash}_1(A) \neq \text{hash}_1(B)$

Can $f(\text{hash}_1(A)) \neq f(\text{hash}_1(B))$



$$P[f(\text{hash}_1(A)) = f(\text{hash}_1(B)) \text{ when } A \neq B]$$

$$= \frac{1}{M}$$

$$X = x_0 \cdot p^{n-1} + x_1 \cdot p^{n-2} \dots x_{n-1} p^0$$

= ①

ok

$$X = x_0 \cdot p^0 + x_1 \cdot p^1 + x_2 p^2 \dots x_{n-1} p^{n-1}$$

= ②

②

Polynomial Rolling Hash

(s) string



$$\begin{aligned}\text{hash}(s) &= \left[s[0] + s[1] \cdot p + s[2] \cdot p^2 + \dots + s[n-1] \cdot p^{n-1} \right] \bmod m \\ &= \left[\sum_{i=0}^{n-1} s[i] \cdot p^i \right] \bmod \underline{\underline{m}},\end{aligned}$$

$$s[0] = s_0 - 'a' + 1$$

$$S = s_0 s_1 s_2 \dots s_{n-1}$$

$$\text{hash}(S) = H$$

$$S + \text{ch}$$

ch = some character

eg: $S = \text{aab}$

$$S + 'c' = \text{aabc}$$

$$\text{hash}(S + \text{ch})$$

$$= s_0 \cdot p^0 + s_1 \cdot p^1 + \dots + s_{n-1} \cdot p^{n-1} + \text{ch} \cdot p^n$$

$$= H + \text{ch} \cdot p^n$$

$$\text{hash}(s) = H$$

$$\text{hash}(ch + s)$$

$$\text{eg: } S = \text{aab}$$

$$'c' + S = \text{caab}$$

$$\begin{aligned} ch + s &= ch \cdot p^0 + s_0 \cdot p^1 + \dots + s_{n-1} \cdot p^n \\ &= ch \cdot p^0 + p \left[s_0 \cdot p^0 + s_1 \cdot p^1 + \dots + s_{n-1} \cdot p^{n-1} \right] \\ &= ch + pH \end{aligned}$$

Why Rolling?

finding hash of s once
takes $O(n)$ time



✓ Hash(s) = H , what is Hash($s + x$), x = character

$O(1)$

○ $(H + xp^n) \% m$

• Hash(s) = H , what is Hash($x + s$), x = character

$O(1)$

○ $(x + Hp) \% m$

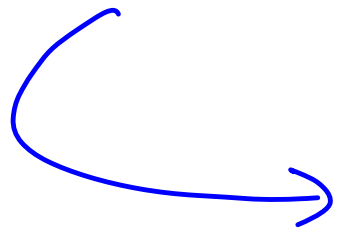
✓✓ How about calculating the Hash of a substring quickly? ==

✓✓ How about just comparing two strings?

$$S = s_0 \ s_1 \ s_2 \ \dots \ s_{n-1}$$

$$\underline{\text{hash}(s)} = [s_0 \cdot p^0 + s_1 \cdot p^1 + \dots + s_{n-1} \cdot p^{n-1}] \% m$$

hash (s [L to R]) quickly

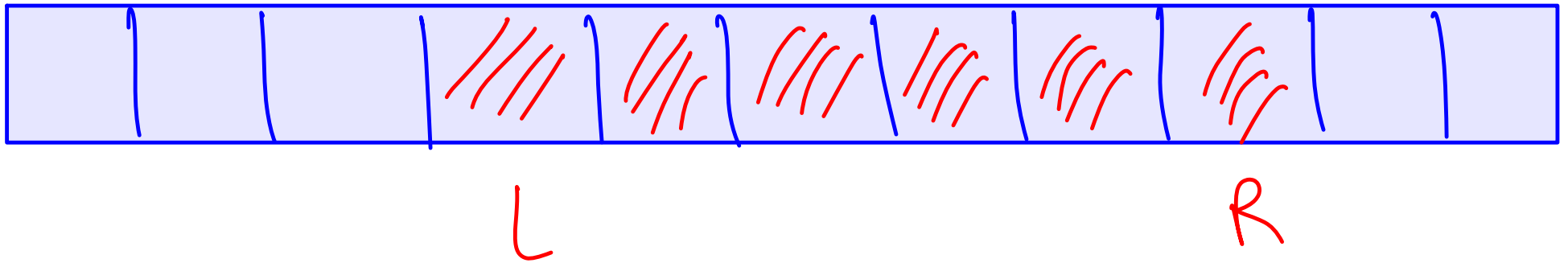


$$\underline{[s_L \cdot p^0 + s_{L+1} \cdot p^1 + \dots + s_R \cdot p^{R-L}] \% m}$$

Compute prefix hashes

$$p\text{-hash}[i] = \text{hash}(s[0 \text{ to } i])$$

$$p\text{-hash}[i] = s_0 \cdot p^0 + s_1 \cdot p^1 + \dots + s_i \cdot p^i$$



$$p\text{-hash}(R) = s_0 \cdot p^0 + s_1 \cdot p^1 + \dots + s_R \cdot p^R$$

$$p\text{-hash}(L-1) = s_0 \cdot p^0 + s_1 \cdot p^1 + \dots + s_{L-1} \cdot p^{L-1}$$

$$p\text{-hash}(R) - p\text{-hash}(L-1)$$

$$\text{have} = s_L \cdot p^L + s_{L+1} \cdot p^{L+1} + \dots + s_R \cdot p^{R+1}$$

$$\text{want} = s_L \cdot p^0 + s_{L+1} \cdot p^1 + \dots + s_R \cdot p^{R-L}$$

divide have by p^L

hash (S (L to R))

$$\left[\frac{\overset{O(1)}{\text{p-hash}[R]} - \overset{O(1)}{\text{p-hash}[L-1]}}{\underset{\text{p}^2}{\text{p}^2}} \right] \% m$$

$O(1 \log m)$

fermat's theorem $\rightarrow a^{m-1} \equiv 1 \pmod m$
when m is prime

and $a^{-1} = a^{m-2} \pmod m$

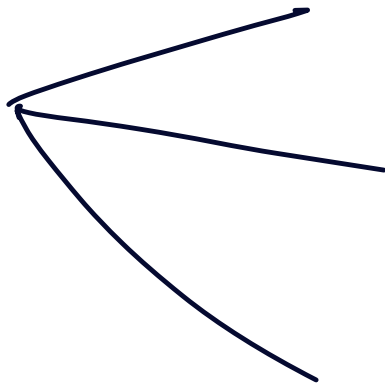
$$A = [a_0 \ a_1 \ \dots \ a_{n-1}]$$

$$B = [b_0 \ b_1 \ \dots \ b_{n-1}]$$

$$1 \leq N, Q \leq 10^5$$

Q queries

L, R



=

if equal (simple hashing)

>

if $A(\text{sub}) > B(\text{sub})$

<

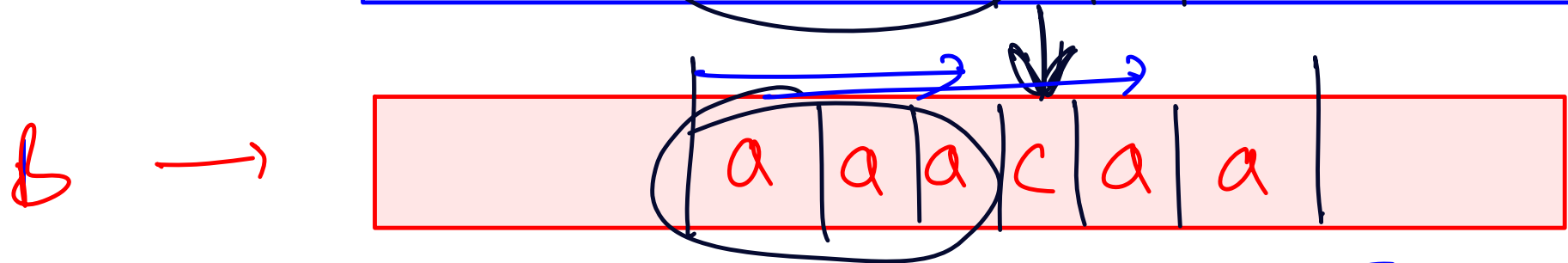
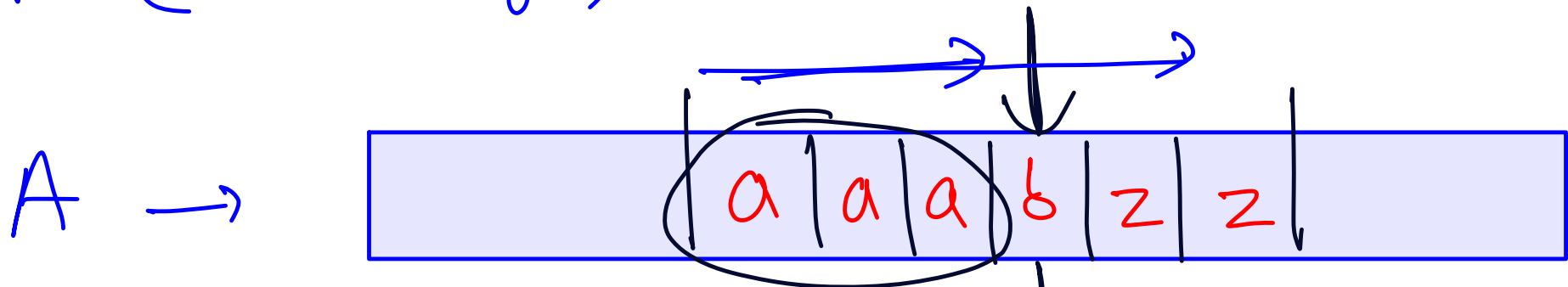
if $A(\text{sub}) < B(\text{sub})$

logn

aa8 < aac

caa > bzz

if $(x > y)$ then $\text{hash}(x) > \text{hash}(y)$



$$\text{hash}[A[l \text{ to } i]] = \text{hash}[B[l \text{ to } i]] \begin{matrix} \nearrow T \\ \searrow F \end{matrix}$$

$$A = B \implies \text{hash}(A) = \text{hash}(B)$$

But

$$A > B \not\Rightarrow \text{hash}(A) > \text{hash}(B)$$

$$A > B \implies \text{hash}(A) \neq \text{hash}(B)$$

A a b a d a c d a a a a c b e f g h

B a s a d a c a a a a a c b e f g h

Pitfalls? $m \rightarrow$ prime $10^9 + 7$
 $\rightarrow$ large $10^9 + 9$



- $A \neq B$ but $\text{Hash}(A) == \text{Hash}(b)$ } one comparison
 - Probability = $1 / m$

- Comparing 50 such strings, probability of a collision = $50 / m$

approx

- ~~Let's look at an example problem to see how it fails~~
 - ~~Substring comparison problem~~

- Solution????

$\rightarrow$ multiple hashes

$$P(\text{fail}) = \frac{1}{m} \quad P(\text{pass}) = \frac{m-1}{m}$$

k comparisons and any 1 fails

$$P(\text{any 1 fails in } k \text{ comparisons})$$

$$= 1 - P(\text{all of them pass})$$

$$= 1 - \left[\frac{m-1}{m} \right]^k$$

$$m = 10^9$$

$$k = 10^5$$

$$= 1 - \left[\frac{10^9 - 1}{10^9} \right]^{10^5}$$

$$= 9.9995e-5 = 9.9995 \times 10^{-5}$$

$$= 0.0009995$$

$$\approx \underline{\underline{0.001}}$$

$$\frac{10^5}{10^9} \approx \underline{\underline{0.0001}}$$

$$\frac{10^{10}}{10^9} > 1$$

global tests $\rightarrow$ each must run in $\leq$

$\hookrightarrow$ input file

$\hookrightarrow$ multiple test cases

1000 GTs

$\hookrightarrow 10^4$ L.T.

$\hookrightarrow 10^3$ comparisons

$m \rightarrow 10^9$

GT₁ $\rightarrow \frac{10^7}{10^9}$

any one GT fails

$$= 1000 \cdot \frac{10^7}{10^9} \geq 1$$

compute 2 hashes

$$\begin{array}{l} \text{hash}_1 \rightarrow \\ \text{hash}_2 \rightarrow \end{array} \left\{ \begin{array}{l} p_1 = 52, \quad m_1 = 10^9 + 7 \\ p_2 = 109, \quad m_2 = 10^9 + 9 \end{array} \right.$$

$$A \stackrel{!}{=} B \quad \text{then} \quad \text{hash}_1(A) = \text{hash}_1(B) \\ = \frac{1}{10^9}$$

$$A \stackrel{!}{=} B \quad \text{then} \quad \text{hash}_2(A) = \text{hash}_2(B) \\ = \frac{1}{10^9}$$

$$A|_0 = B \quad \text{and} \quad \text{hash}_1(A) = \text{hash}_1(B)$$

and

$$\text{hash}_2(A) = \text{hash}_2(B)$$

$$= \frac{1}{m_1} \cdot \frac{1}{m_2} = \frac{1}{m_1 \cdot m_2}$$

1000 GTs

→ 10^4 LTs

→ 10^3 comparisons

$$= \frac{1000 \cdot 10^7}{10^{18}}$$

String Hashing Template



https://github.com/Priyansh19077/CP-Templates/blob/master/String%20Algorithms/String_Hashing.cpp

Problem 1

$$N \leq 10^3$$



set <int> unique subs.

Given a string S of length N, find the number of distinct substrings present in the string.

Examples:

abc => 6

aab => 5

aaa => 3

```
for (int i = 0 ; i < n ; i++)
```

```
for (int j = i , j < n ; j++)
```

```
hash = gethash(i, j) }
```

```
uniquesubs.insert(hash)
```

```
cout << len(uniquesubs)
```



Problem 2

Given a string S of length N ($1 \leq N \leq 10^5$) and Q ($1 \leq Q \leq 10^5$) queries where each query contains 2 integers L and R ($1 \leq L \leq R \leq N$), find out for each query if the substring from L to R is a palindrome.

Example:

abccbd, $q_1 = [3, 4] \Rightarrow \text{Yes}$, $q_2 = [1, 5] \Rightarrow \text{No}$, $q_3 = [2, 4] \Rightarrow \text{Yes}$



Problem 3

Given a string S of length N ($1 \leq N \leq 10^5$), find out the longest substring which occurs at least twice in the string.

Examples:

abcdabcd $\Rightarrow$ 4 (abcd)

abbbbbbbd $\Rightarrow$ 5 (bbbbbb)

abccab $\Rightarrow$ 2 (ab)



Problem 4

Given a string S of length N ($1 \leq N \leq 10^5$), find out the longest palindromic substring.

Examples:

abbbbc $\Rightarrow$ 4 (bbbb)

abbabba $\Rightarrow$ 7 (abbabba)

abacde $\Rightarrow$ 3 (aba)



Files

master

Go to file

Common Techniques

Graph Algorithms

Number Theory

Range Queries

String Algorithms

Prefix_Function.cpp

String_Hashing.cpp

Suffix_array.cpp

Trie.cpp

ZAlgrithm.cpp

TemplateStuff

Tree Algorithms

.gitignore

CP-Templates / String Algorithms / String_Hashing.cpp



Priyansh19077 corrected typo

866cef1 · last year

History

Code

Blame

46 lines (46 loc) · 1.77 KB

Raw



```
1 struct Hashing{
2     string s;
3     int n;
4     int primes;
5     vector<ll> hashPrimes = {1000000009, 1000000007};
6     const ll base = 31;
7     vector<vector<ll>> hashValues;
8     vector<vector<ll>> powersOfBase;
9     vector<vector<ll>> inversePowersOfBase;
10    Hashing(string a){
11        primes = sz(hashPrimes);
12        hashValues.resize(primes);
13        powersOfBase.resize(primes);
14        inversePowersOfBase.resize(primes);
15        s = a;
16        n = s.length();
17        for(int i = 0; i < sz(hashPrimes); i++) {
18            powersOfBase[i].resize(n + 1);
19            inversePowersOfBase[i].resize(n + 1);
20            powersOfBase[i][0] = 1;
21            for(int j = 1; j <= n; j++){
22                powersOfBase[i][j] = (base * powersOfBase[i][j - 1]) % hashPrimes[i];
23            }
24            inversePowersOfBase[i][n] = mmInvprime(powersOfBase[i][n], hashPrimes[i]);
25            for(int j = n - 1; j >= 0; j--){
26                inversePowersOfBase[i][j] = mod_mul(inversePowersOfBase[i][j + 1], base, hashPrimes[i]);
27            }
28            for(int i = 0; i < sz(hashPrimes); i++) {
29                hashValues[i].resize(n);
30                for(int j = 0; j < n; j++){
31                    hashValues[i][j] = ((s[j] - 'a' + 1LL) * powersOfBase[i][j]) % hashPrimes[i];
32                    hashValues[i][j] = (hashValues[i][j] + (j > 0 ? hashValues[i][j - 1] : 0LL)) % hashPrimes[i];
33                }
34            }
35        }
36    }
37    vector<ll> substringHash(int l, int r){
38        vector<ll> hash(primes);
39        for(int i = 0; i < primes; i++){
40            ll val1 = hashValues[i][r];
41            ll val2 = l > 0 ? hashValues[i][l - 1] : 0LL;
42            hash[i] = mod_mul(mod_sub(val1, val2, hashPrimes[i]), inversePowersOfBase[i][l], hashPrimes[i]);
43        }
44        return hash;
45    }
46 }
```

$$\text{hash}[i] = \frac{(\text{hash_value}(i)[r] - \text{hash_value}(i)[l-1]) \% m}{\text{base}[i][l]}$$